

```
# Visualize the results.
title = "Results of Rolling Two D6 Dice 1,000 Times"
labels = {'x': 'Result', 'y': 'Frequency of Result'}
fig = px.bar(x=pos_results, y=frequencies, title=title, labels=labels)
fig.show()
```

After creating two instances of `Die`, we roll the dice and calculate the sum of the two dice for each roll ❶. The smallest possible result (2) is the sum of the smallest number on each die. The largest possible result (12) is the sum of the largest number on each die, which we assign to `max_result` ❷. The variable `max_result` makes the code for generating `pos_results` much easier to read ❸. We could have written `range(2, 13)`, but this would work only for two D6 dice. When modeling real-world situations, it's best to write code that can easily model a variety of situations. This code allows us to simulate rolling a pair of dice with any number of sides.

After running this code, you should see a chart that looks like Figure 15-14.

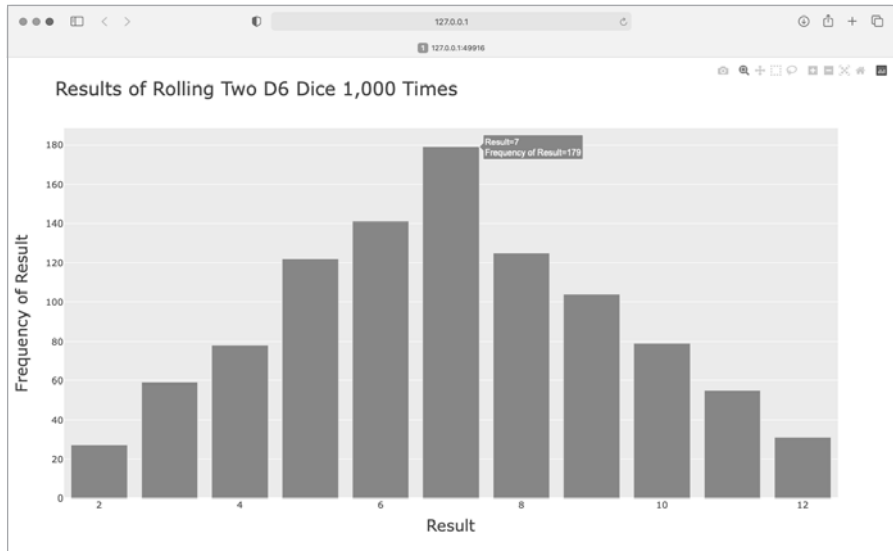


Figure 15-14: Simulated results of rolling two six-sided dice 1,000 times

This graph shows the approximate distribution of results you're likely to get when you roll a pair of D6 dice. As you can see, you're least likely to roll a 2 or a 12 and most likely to roll a 7. This happens because there are six ways to roll a 7: 1 and 6, 2 and 5, 3 and 4, 4 and 3, 5 and 2, and 6 and 1.

Further Customizations

There's one issue that we should address with the plot we just generated. Now that there are 11 bars, the default layout settings for the *x*-axis leave some of the bars unlabeled. While the default settings work well for most visualizations, this chart would look better with all of the bars labeled.